CompSci 469/469G
Intro to Computer Security
Linux Security Basics, Set-UID Programs

Source

- Computer Security-A Hands on Approach (3rd Edition), Wenliang Du.
  - Referenced Chapters: 1, 2

Default File Permissions

- When a file is created, the default permissions is decided by umask value.
- umask = user file-creation mode mask.
  - tells the system which permission bits should not be set on newly created files
- How to see umask value? umask
- For non-executable file, the initial permission is 0666.

```
Initial  (0666)       rw-  rw-  rw-
                      110  110  110
umask    (0022)       000  010  010
-------------------------------------
Final permission      110  100  100
                      rw-  r--  r--
```

Initial & ~umask

Demo

```
root@52b7cff8b811:/# touch test1
root@52b7cff8b811:/# ls -l test1
-rw-r--r-- 1 root root 0 Sep 11 19:49 test1

root@52b7cff8b811:/# umask 0000
root@52b7cff8b811:/# touch test2
root@52b7cff8b811:/# ls -l test2
-rw-rw-rw- 1 root root 0 Sep 11 19:49 test2

root@52b7cff8b811:/# umask 0777
root@52b7cff8b811:/# touch test3
root@52b7cff8b811:/# ls -l test3
---------- 1 root root 0 Sep 11 19:50 test3
```

Initial & ~umask

Why umask?

- **Security by default**
  - Prevents files from being world-readable or world-writable unless the user explicitly sets it.
  - Example: with umask 0022, new files get 644 (rw-r--r--), so only the owner can write.

sudo

- In Ubuntu 20.04 and later, the root account is locked by default:
- Possible to get root shell, but not best recommended:
  - `sudo -s`
  - `sudo bash`
  - `sudo su`
- Instead use sudo to run individual commands:
  - `sudo apt update`

sudo

- For using sudo, you must be in sudoers list:
  - From root account: usermod –aG sudo bob
- Root can add or remove users from sudo group:
  - Adding: `usermod –aG sudo bob`
  - Removing: `gpasswd -d bob sudo`
- User must set a password to be in sudoers:
  - Login as that user and enter: `passwd`
- How to see sudoers: `getent group sudo`

Password Dilemma

- Changing password means adding an entry to /etc/shadow
  - /etc/shadow contains hashed passwords not plain text
- Permissions of /etc/shadow File:

```
ls -l /etc/shadow
-rw-r----- 1 root shadow 723 Sep 11 20:05 /etc/shadow
```

- How would normal users change their password?
  - Passwd command invoke /bin/passwd program [a compiled binary program]
  - Permissions of /bin/passwd File:

```
ls -l /bin/passwd
-rwsr-xr-x 1 root root 72056 May 30  2024 /bin/passwd
```



SET-UID Privileged Program

- Program marked with a special bit - treated specially when running.
- *Every process has two User IDs.*
  - RUID = the user who launched the program.
    - e.g. If bob runs /bin/passwd, the RUID = bob.
  - EUID = the user whose privileges the program is running with.

- Normally, EUID = RUID.
- But if a file has the setuid bit set, then EUID = file owner's UID.
- For /bin/passwd (owned by root), EUID = root.

** Privileged programs are essential but they often become attack targets

Demo: Turn a Program into Set-UID

```
cp /bin/cat ./mycat
bob@52b7cff8b811:~$ ls -l mycat
-rwxr-xr-x 1 bob bob 67928 Sep 11 20:40 mycat

bob@52b7cff8b811:~$ ./mycat /etc/shadow
./mycat: /etc/shadow: Permission denied
```

Making a copy of cat program to be safe

mycat is not a privileged program

Bob can't access shadow file

```
bob@52b7cff8b811:~$ sudo chown root ./mycat
bob@52b7cff8b811:~$ sudo chmod 4755 ./mycat
bob@52b7cff8b811:~$ ls -l ./mycat
-rwsr-xr-x 1 root bob 67928 Sep 11 20:40 ./mycat

bob@52b7cff8b811:~$ ./mycat /etc/shadow
root:*:20305:0:99999:7:::
daemon:*:20305:0:99999:7:::
bin:*:20305:0:99999:7:::
```

Chown command: change ownership of a file

To set set-uid bit

mycat Become a privileged program

Now Bob can access shadow file.
We didn't change the file's permissions; instead, we made the program that accesses it run with elevated privileges.